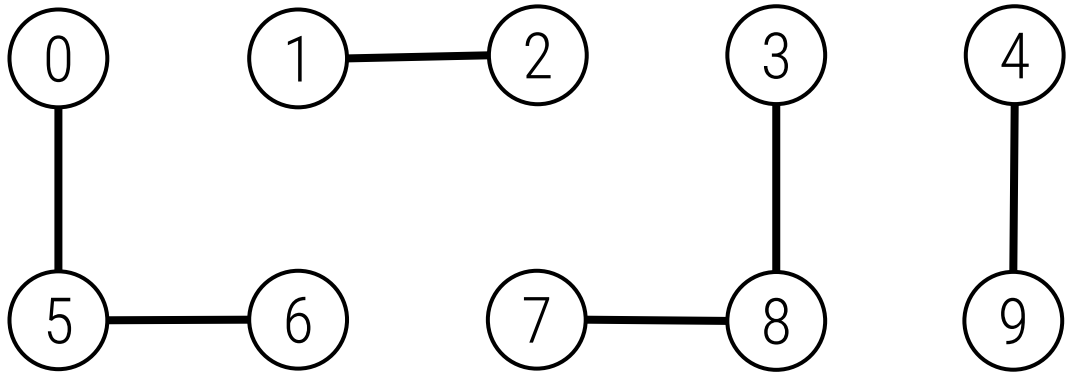
Estructura UFDS

Alberto Verdejo
Facultad de Informática (UCM)

Plan de formación OIE

Relación de equivalencia

- Queremos representar una relación de equivalencia R:
  - Reflexiva: $a\ R\ a$
  - Simétrica: $a\ R\ b \Rightarrow b\ R\ a$
  - Transitiva: $a\ R\ b \wedge b\ R\ c \Rightarrow a\ R\ c$



Partición:

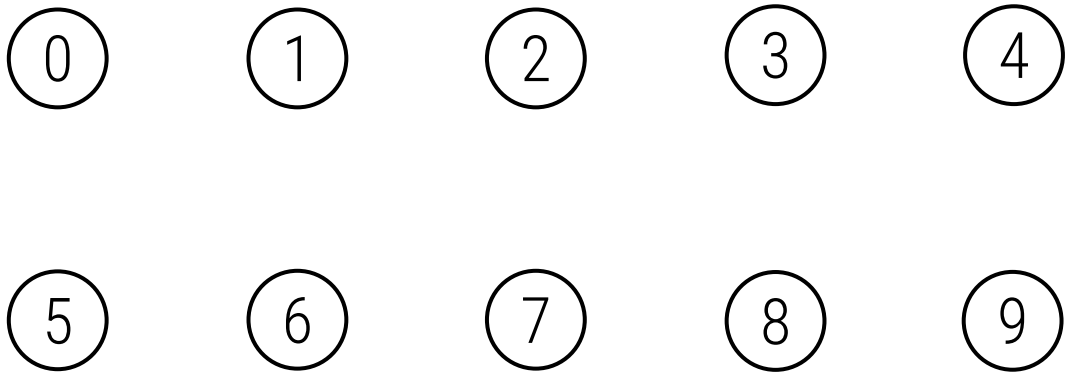
{ 0, 5, 6 }

{ 1, 2 }

{ 3, 7, 8 }

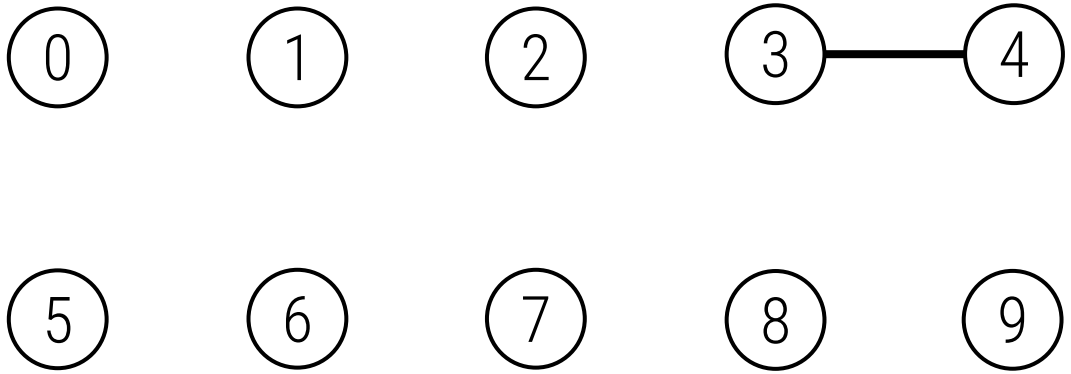
{ 4, 9 }

Relación de equivalencia dinámica



Relación de equivalencia dinámica

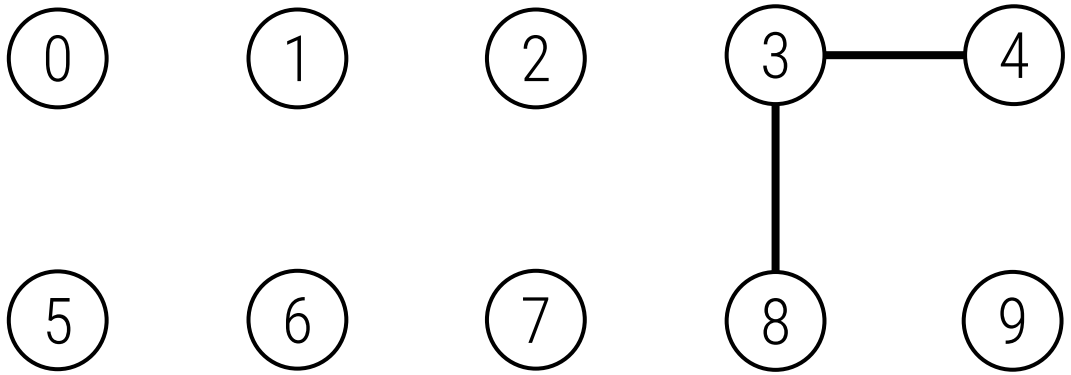
$4 \; R \; 3$



Relación de equivalencia dinámica

$4\ R\ 3$

$3\ R\ 8$

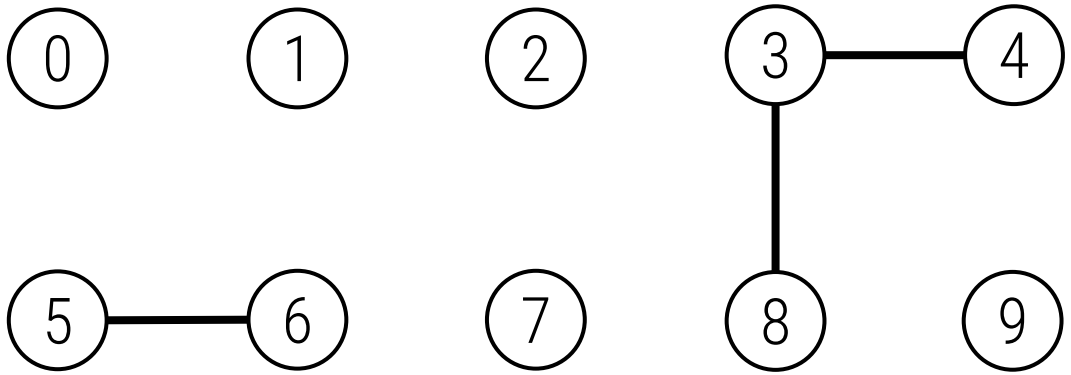


Relación de equivalencia dinámica

$4\ R\ 3$

$3\ R\ 8$

$6\ R\ 5$



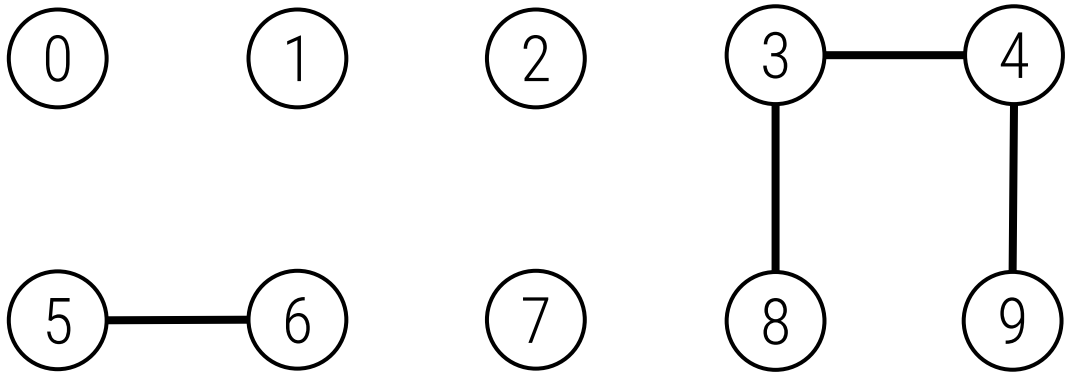
Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

6 *R* 5

9 *R* 4



Relación de equivalencia dinámica

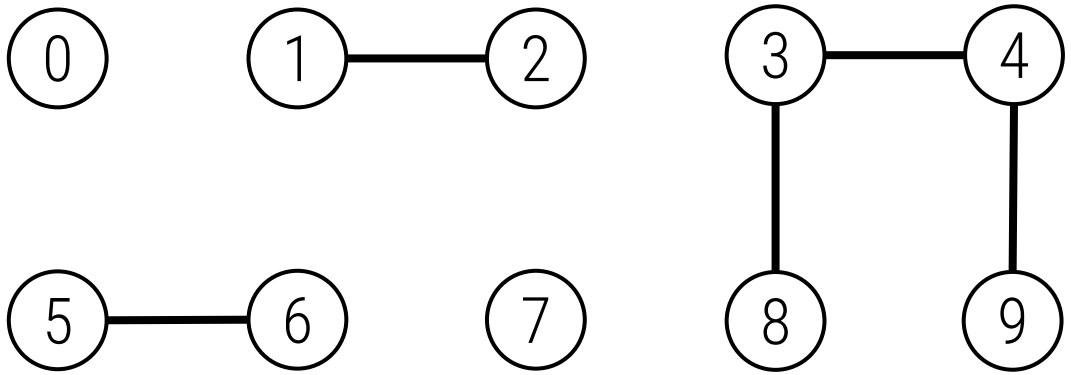
$4\ R\ 3$

$3\ R\ 8$

$6\ R\ 5$

$9\ R\ 4$

$2\ R\ 1$



Relación de equivalencia dinámica

4 *R* 3

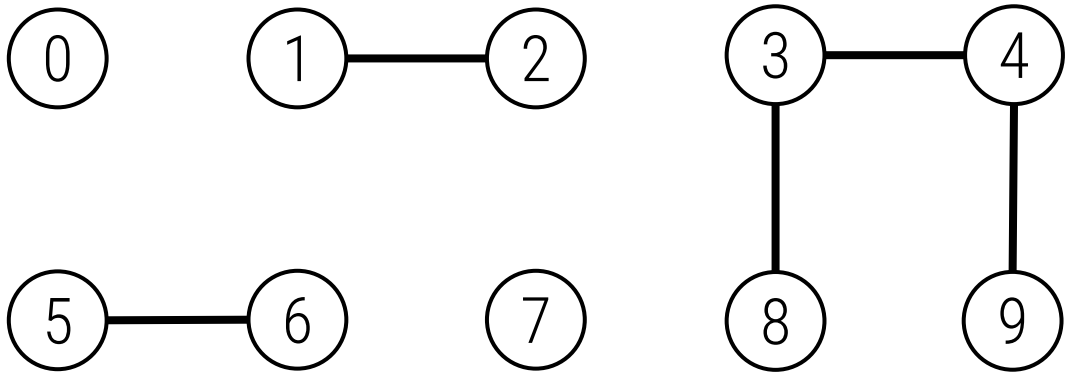
3 *R* 8

6 *R* 5

9 *R* 4

2 *R* 1

¿ 8 *R* 9 ? ✓



Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

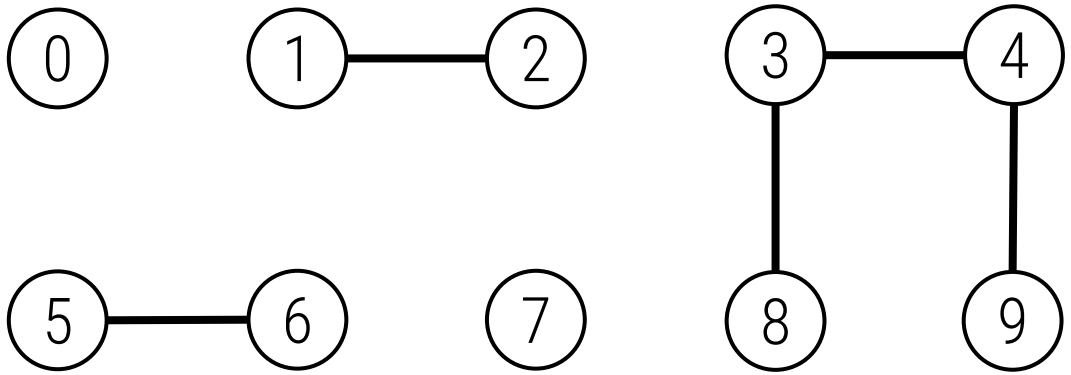
6 *R* 5

9 *R* 4

2 *R* 1

¿ 8 *R* 9 ? ✓

¿ 5 *R* 7 ? ✗



Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

6 *R* 5

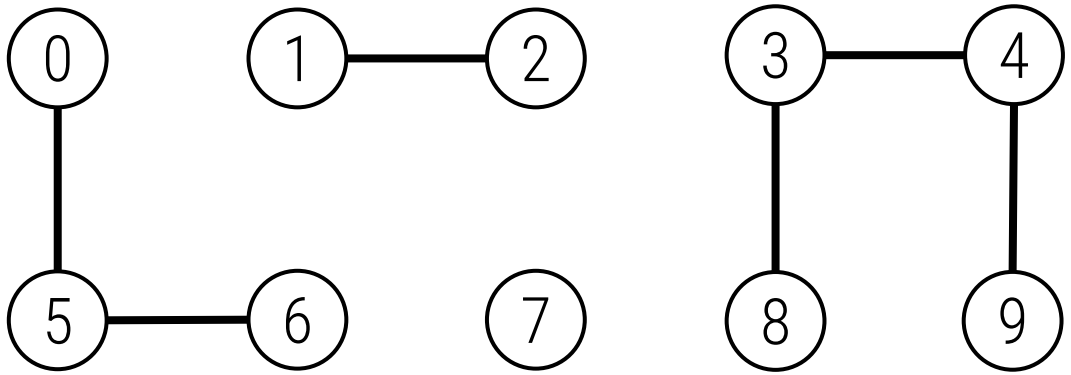
9 *R* 4

2 *R* 1

¿ 8 *R* 9 ? ✓

¿ 5 *R* 7 ? ✗

5 *R* 0



Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

6 *R* 5

9 *R* 4

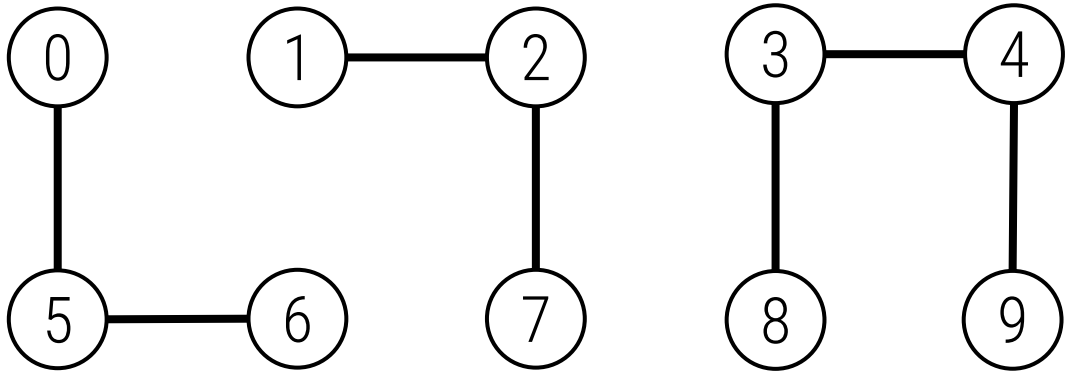
2 *R* 1

¿ 8 *R* 9 ? ✓

¿ 5 *R* 7 ? ✗

5 *R* 0

7 *R* 2



Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

6 *R* 5

9 *R* 4

2 *R* 1

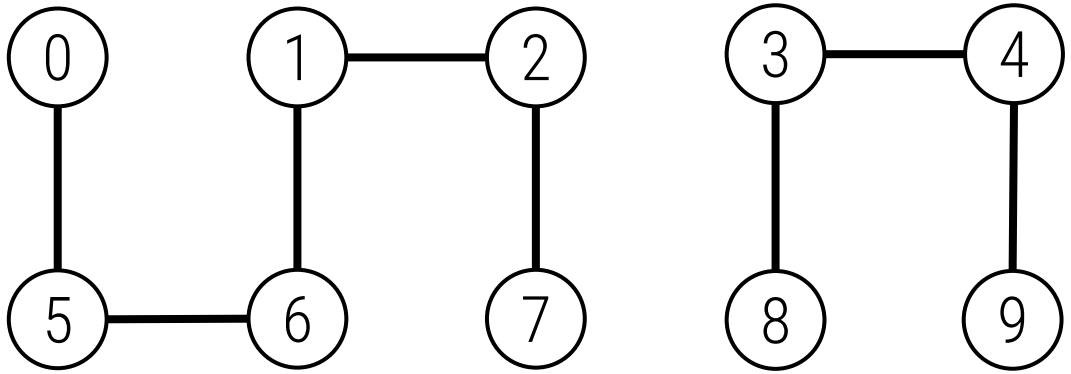
¿ 8 *R* 9 ? ✓

¿ 5 *R* 7 ? ✗

5 *R* 0

7 *R* 2

6 *R* 1



Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

6 *R* 5

9 *R* 4

2 *R* 1

¿ 8 *R* 9 ? ✓

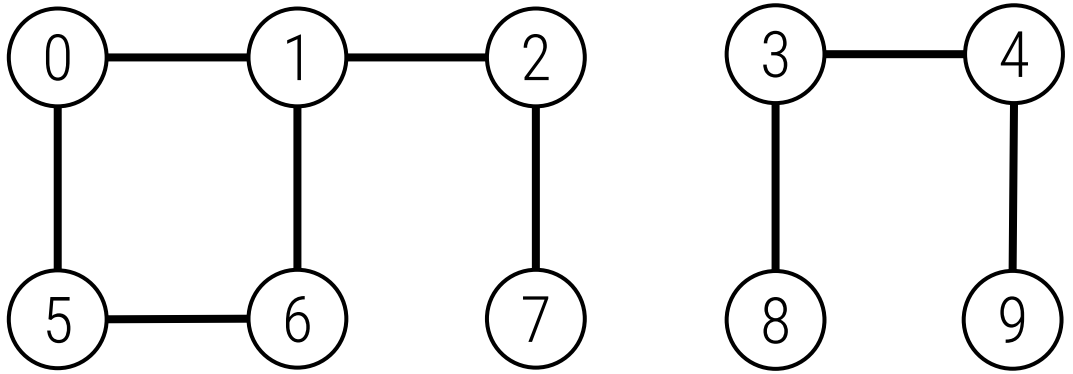
¿ 5 *R* 7 ? ✗

5 *R* 0

7 *R* 2

6 *R* 1

1 *R* 0



Relación de equivalencia dinámica

4 *R* 3

3 *R* 8

6 *R* 5

9 *R* 4

2 *R* 1

¿ 8 *R* 9 ? ✓

¿ 5 *R* 7 ? ✗

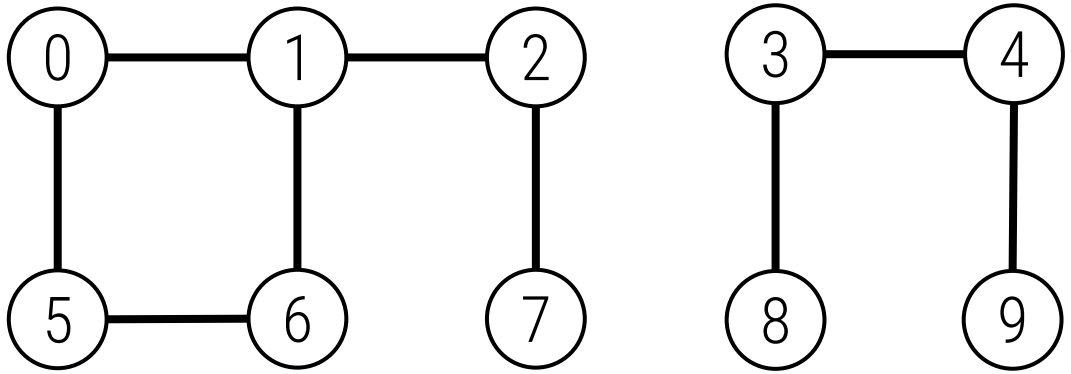
5 *R* 0

7 *R* 2

6 *R* 1

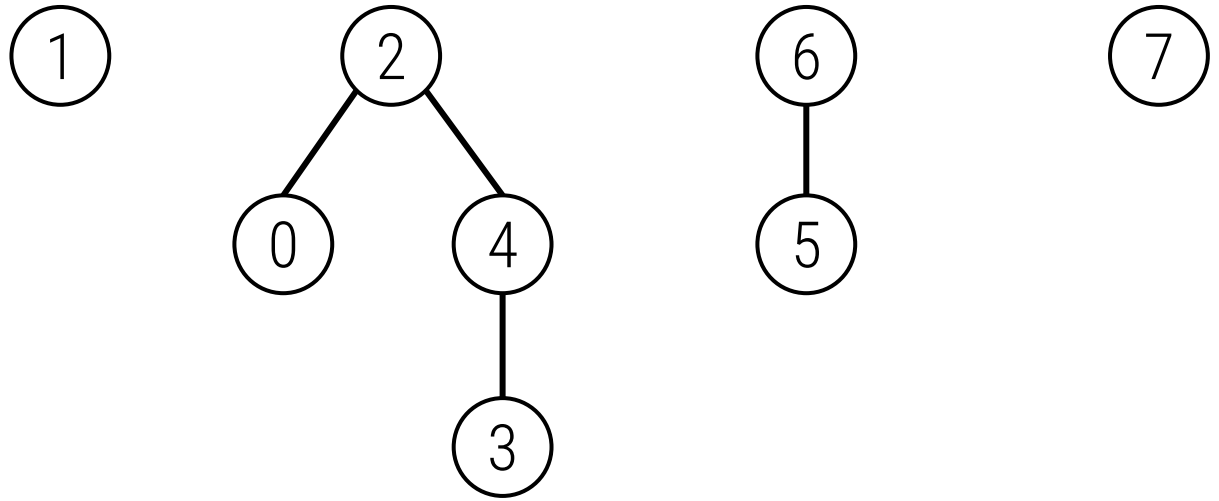
1 *R* 0

¿ 5 *R* 7 ? ✓



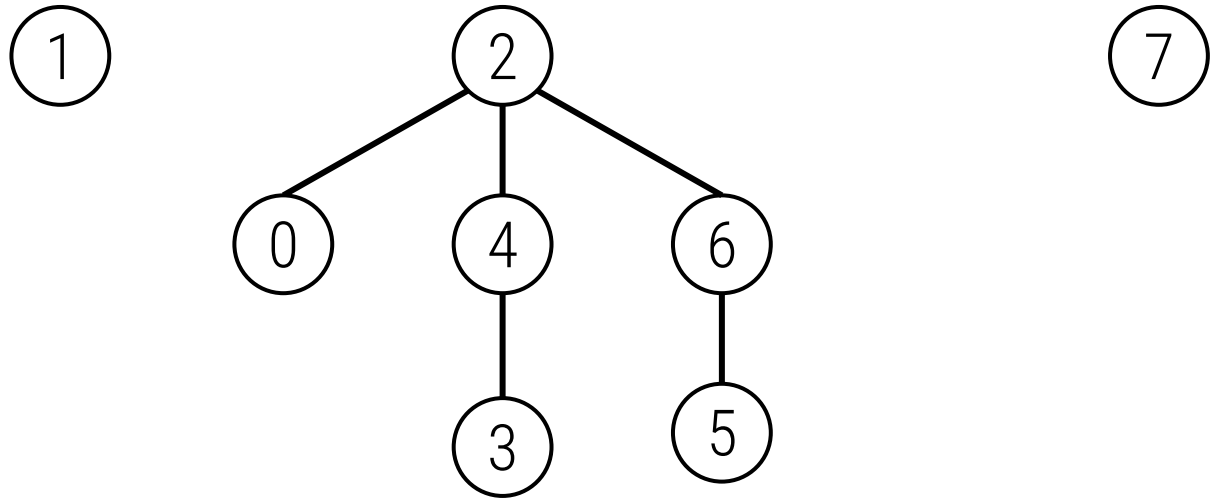
Unión rápida

- Cada conjunto se representa mediante un árbol.



Unión rápida

- Cada conjunto se representa mediante un árbol.

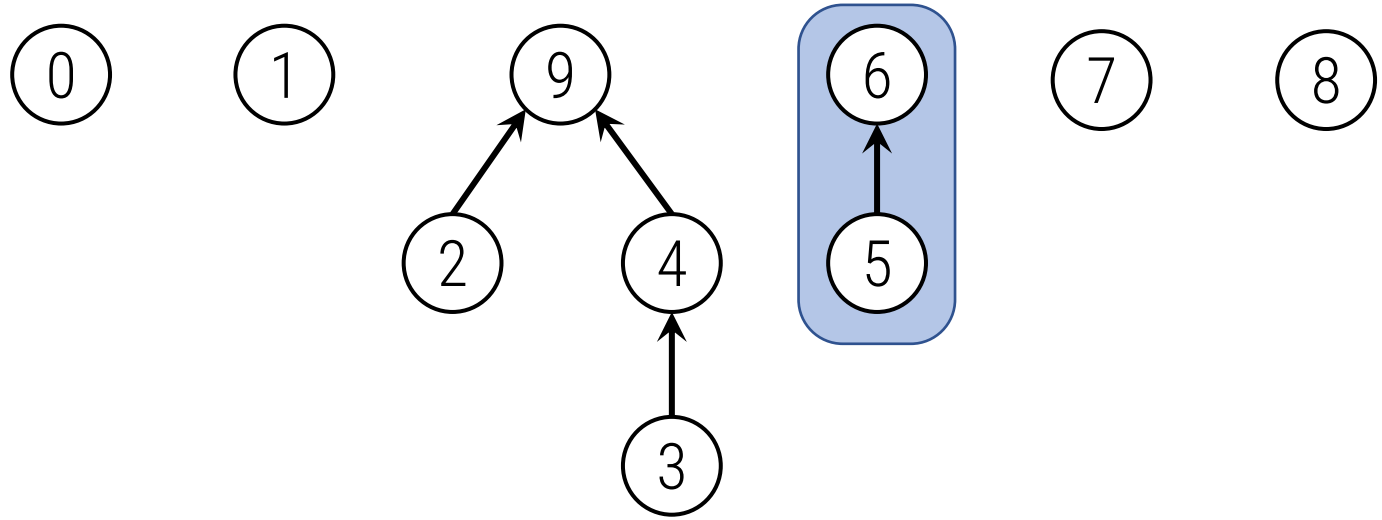


Unión rápida

- Cada conjunto se representa mediante un árbol. `p[i]` es el padre de `i`

	0	1	2	3	4	5	6	7	8	9
p[]	0	1	9	4	9	6	6	7	8	9

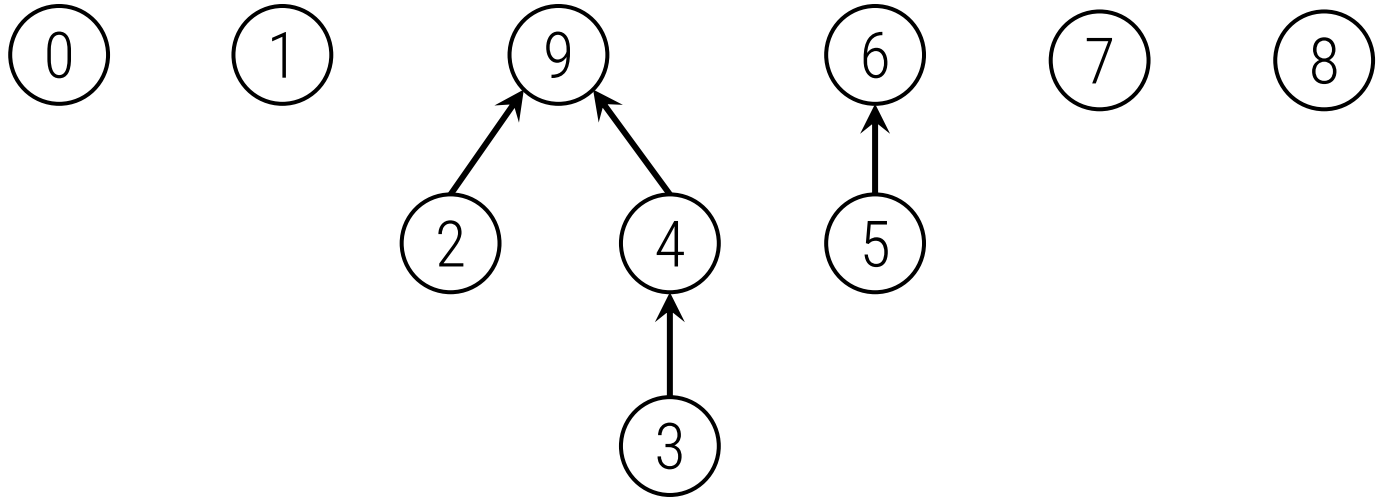
{ 0 }
{ 1 }
{ 2, 3, 4, 9 }
{ 5, 6 }
{ 7, 8 }



Unión rápida

```
unir(3,5)
```

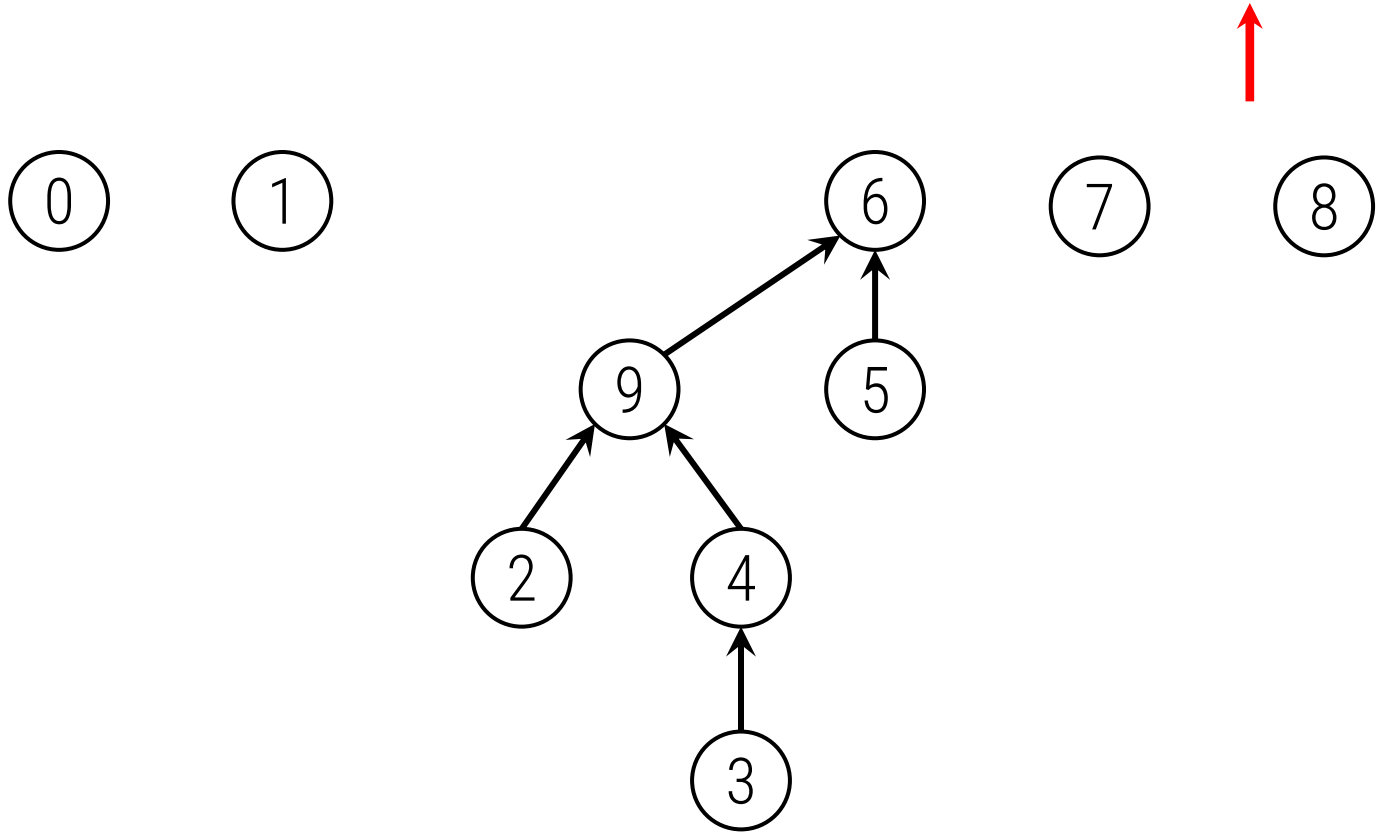
	0	1	2	3	4	5	6	7	8	9
p[]	0	1	9	4	9	6	6	7	8	9



Unión rápida

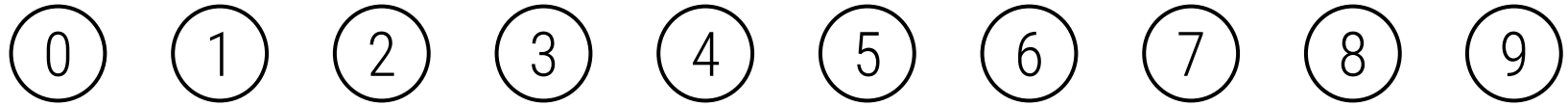
```
unir(3,5)
```

	0	1	2	3	4	5	6	7	8	9
p[]	0	1	9	4	9	6	6	7	8	6



Unión rápida

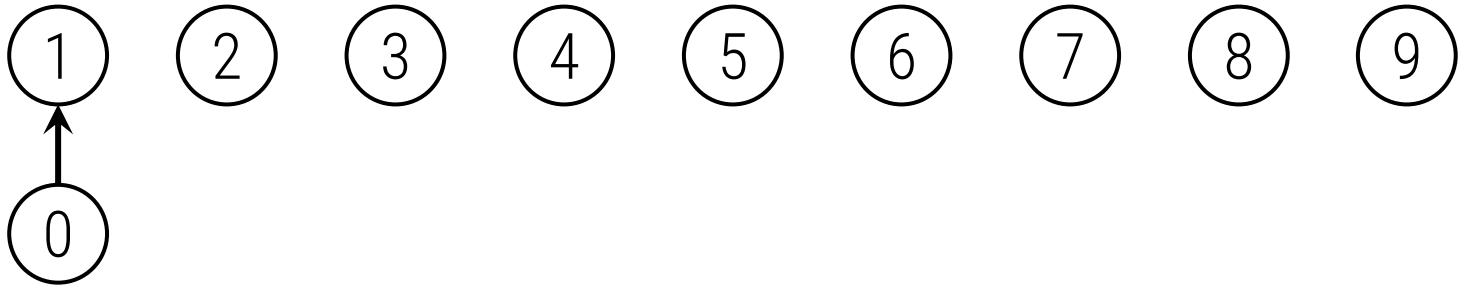
```
unir(0,1)
```



Unión rápida

```
unir(0,1)

unir(0,2)
```

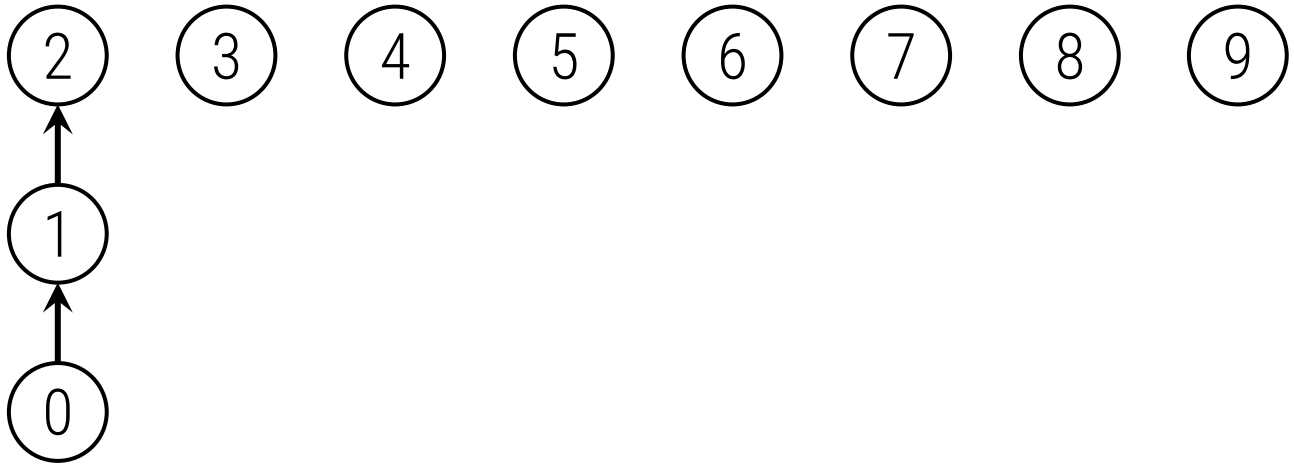


Unión rápida

```
unir(0,1)

unir(0,2)

unir(0,3)
```



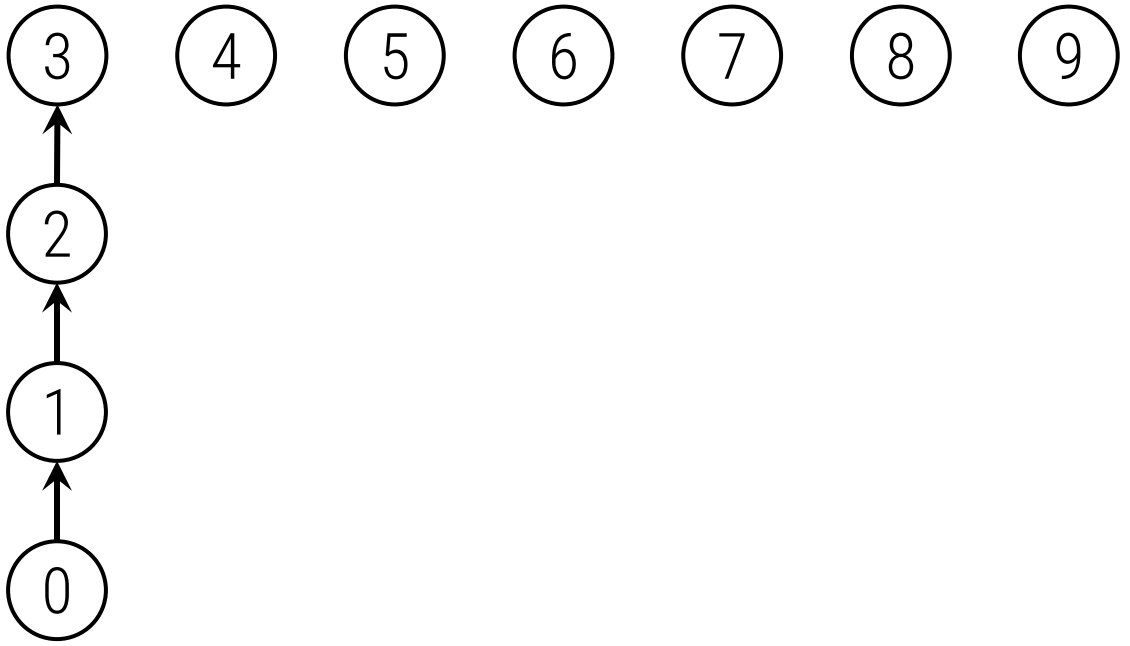
Unión rápida

```
unir(0,1)

unir(0,2)

unir(0,3)

unir(0,4)
```



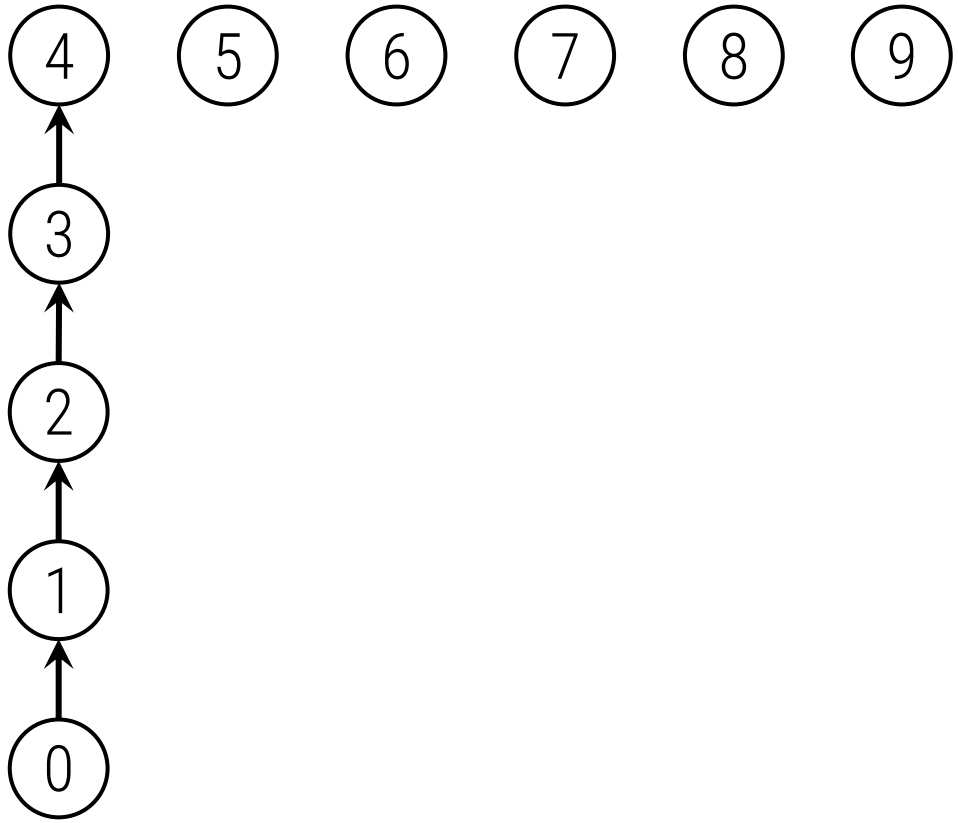
Unión rápida, ¿búsqueda lenta?

```
unir(0,1)

unir(0,2)

unir(0,3)

unir(0,4)
```

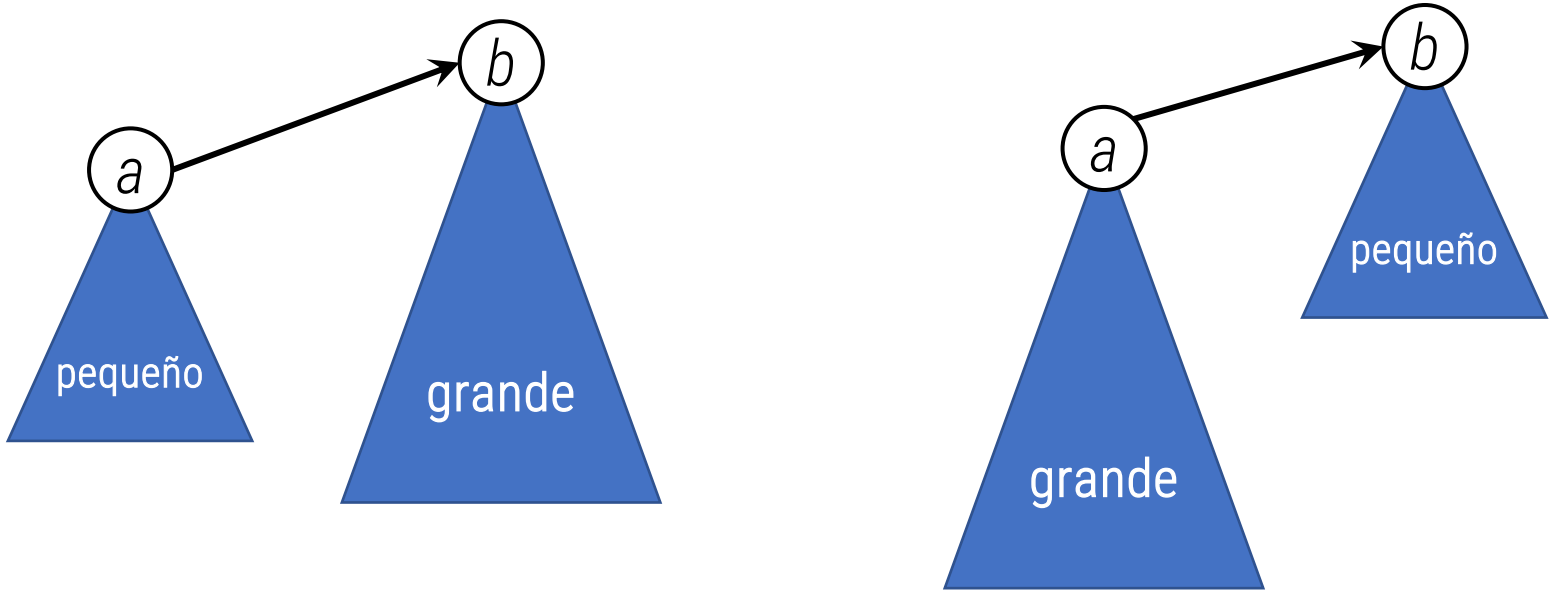


Unión por tamaños

- Vamos a mantener los tamaños de los árboles (número de elementos)
- Al unir, el árbol más pequeño pasa a ser hijo del árbol mayor

Unión rápida

```
unir(a,b)
```

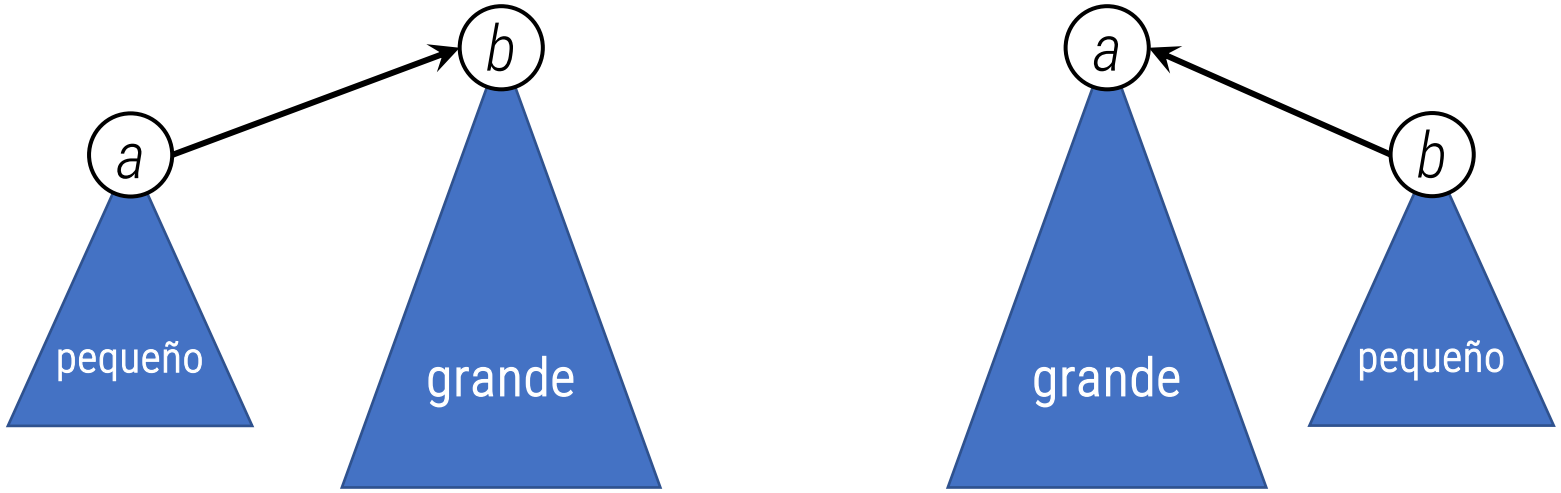


Unión por tamaños

- Vamos a mantener los tamaños de los árboles (número de elementos)
- Al unir, el árbol más pequeño pasa a ser hijo del árbol mayor

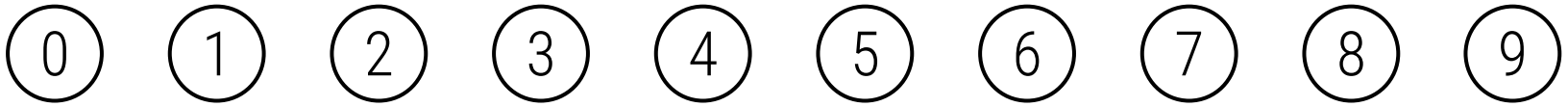
Unión rápida por tamaños

```
unir(a,b)
```



Unión por tamaños

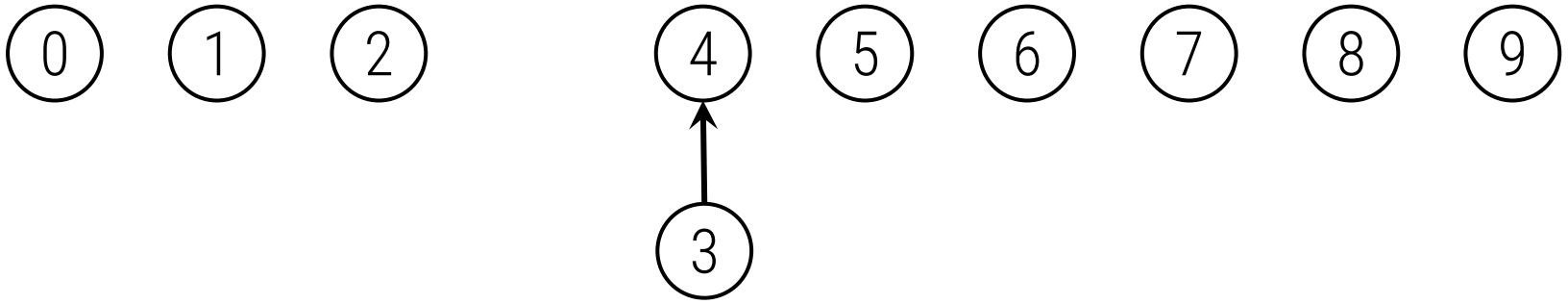
```
unir(4,3)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	3	4	5	6	7	8	9

Unión por tamaños

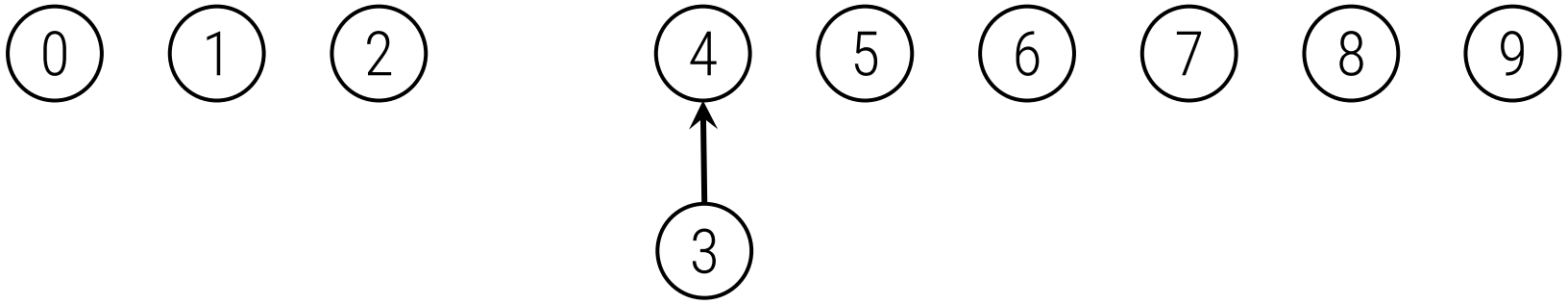
```
unir(4,3)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	5	6	7	8	9

Unión por tamaños

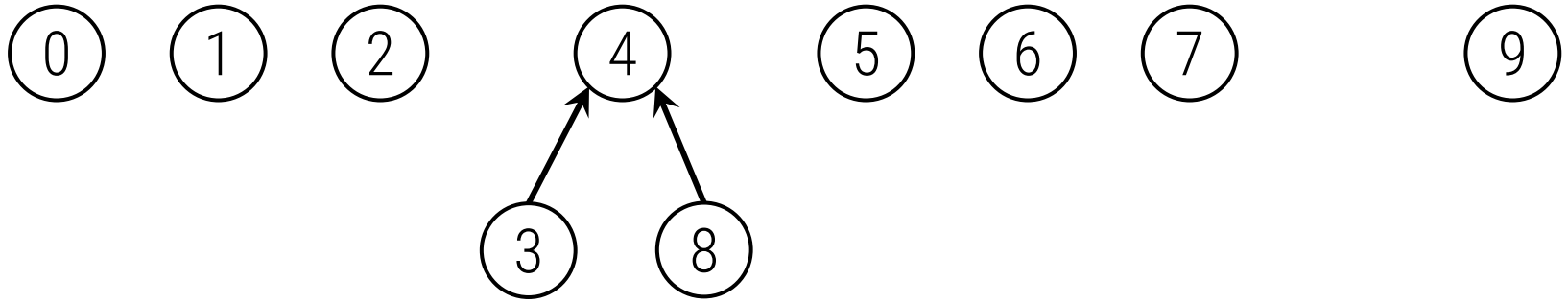
```
unir(3,8)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	5	6	7	8	9

Unión por tamaños

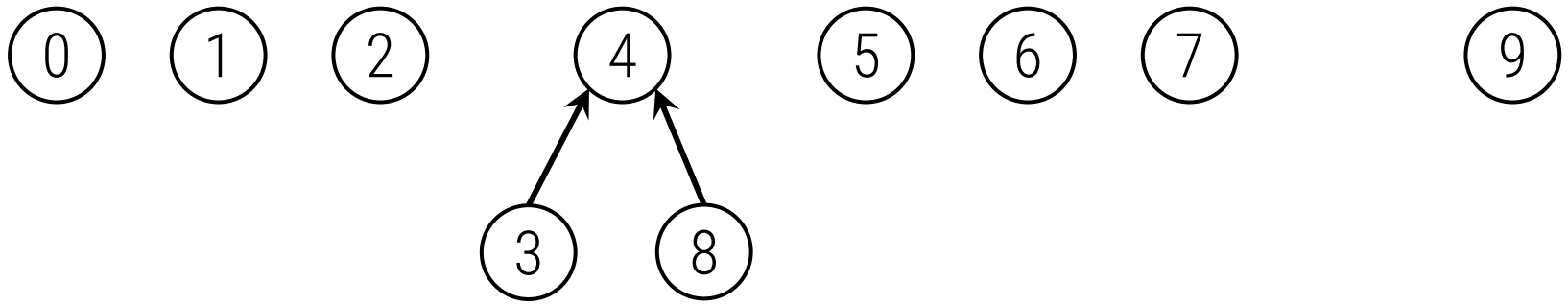
```
unir(3,8)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	5	6	7	4	9

Unión por tamaños

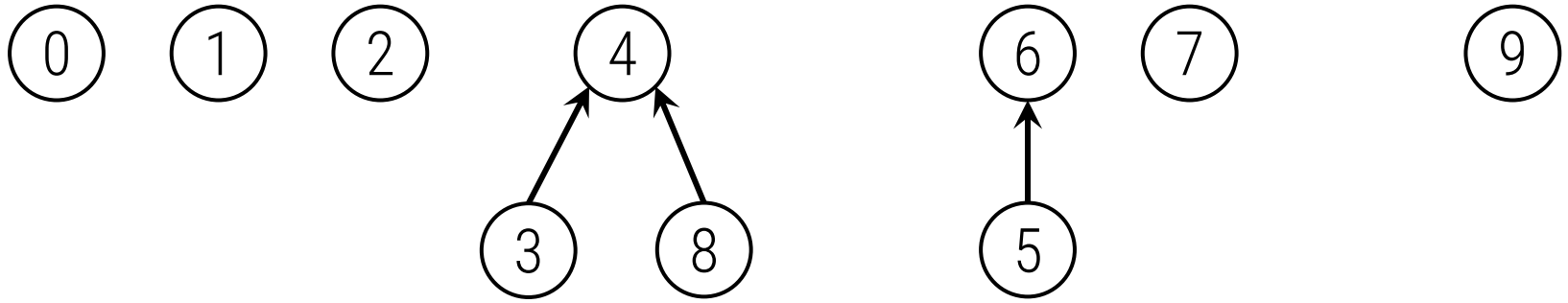
```
unir(6,5)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	5	6	7	4	9

Unión por tamaños

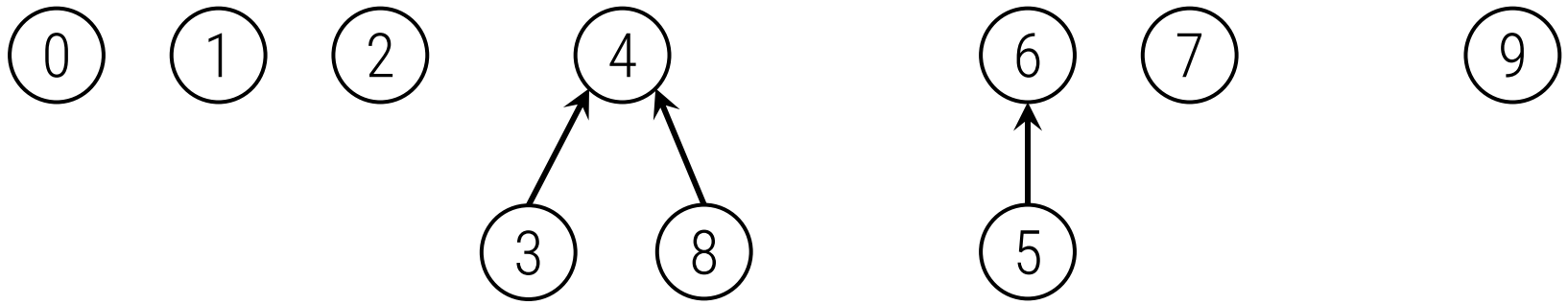
```
unir(6,5)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	6	6	7	4	9

Unión por tamaños

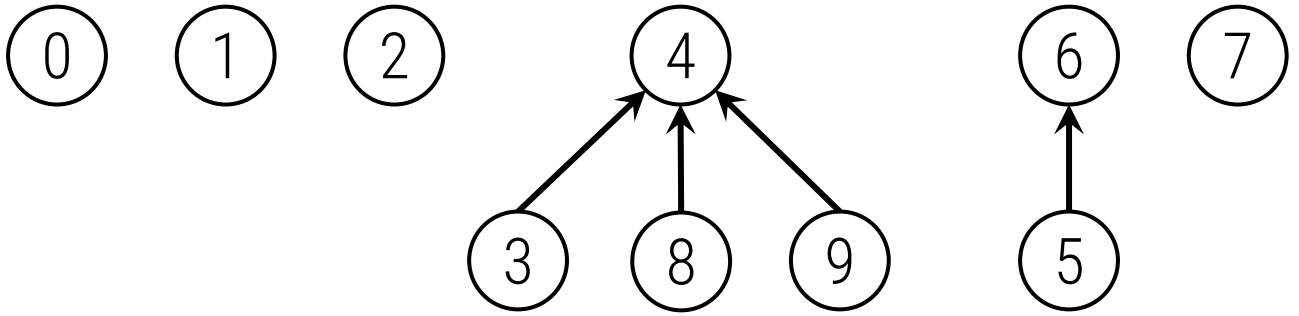
```
unir(9,4)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	6	6	7	4	9

Unión por tamaños

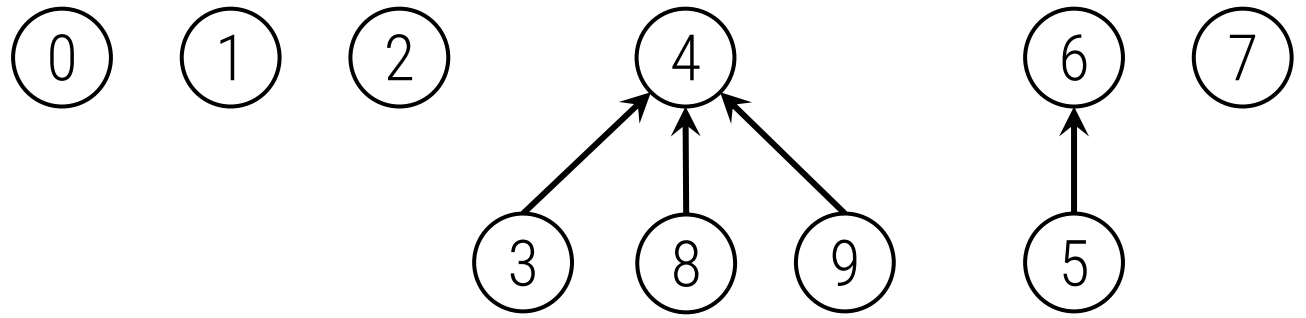
```
unir(9,4)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	6	6	7	4	4

Unión por tamaños

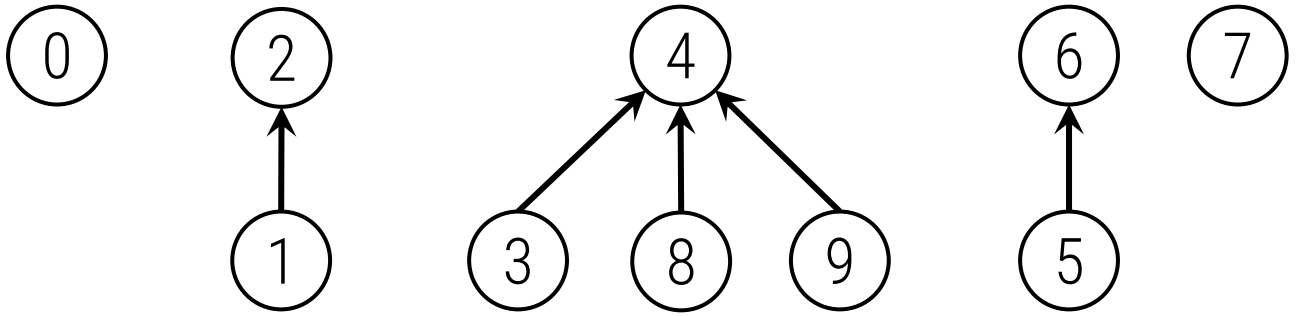
```
unir(2,1)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	1	2	4	4	6	6	7	4	4

Unión por tamaños

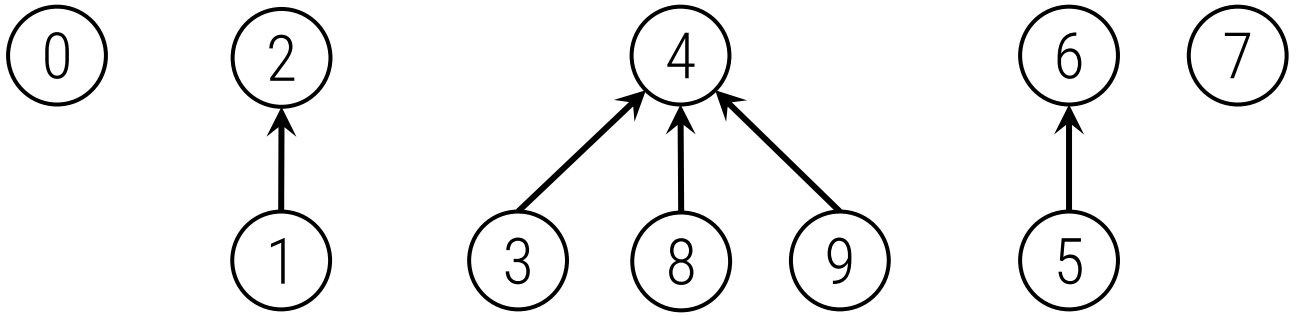
```
unir(2,1)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	2	2	4	4	6	6	7	4	4

Unión por tamaños

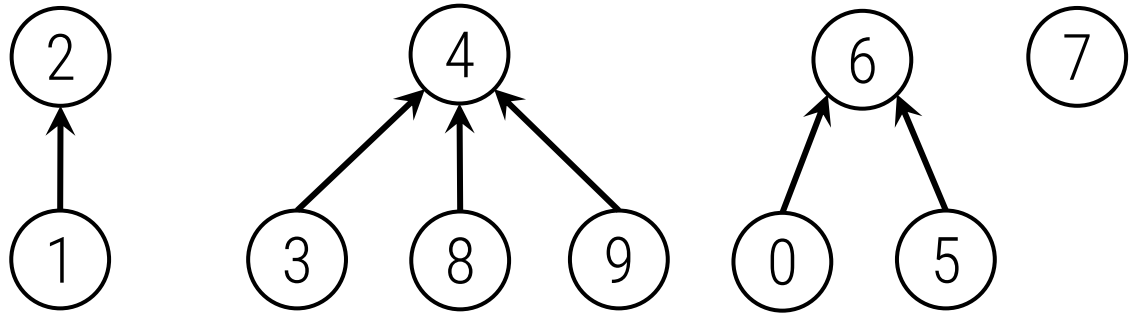
```
unir(5,0)
```



	0	1	2	3	4	5	6	7	8	9
p[]	0	2	2	4	4	6	6	7	4	4

Unión por tamaños

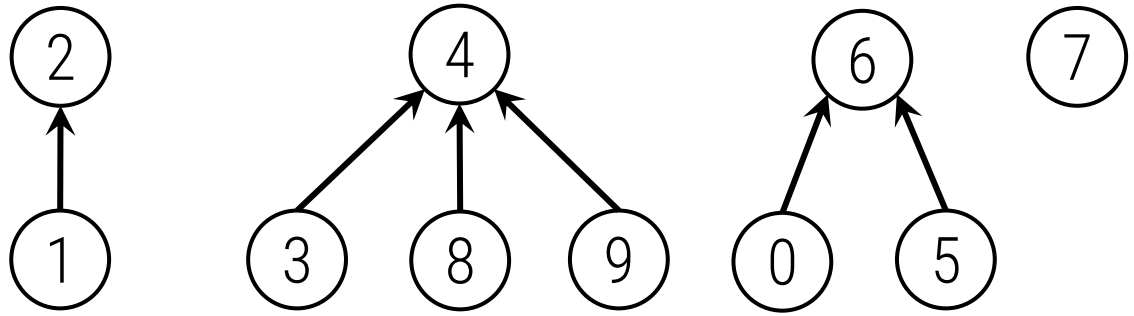
```
unir(5,0)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	2	4	4	6	6	7	4	4

Unión por tamaños

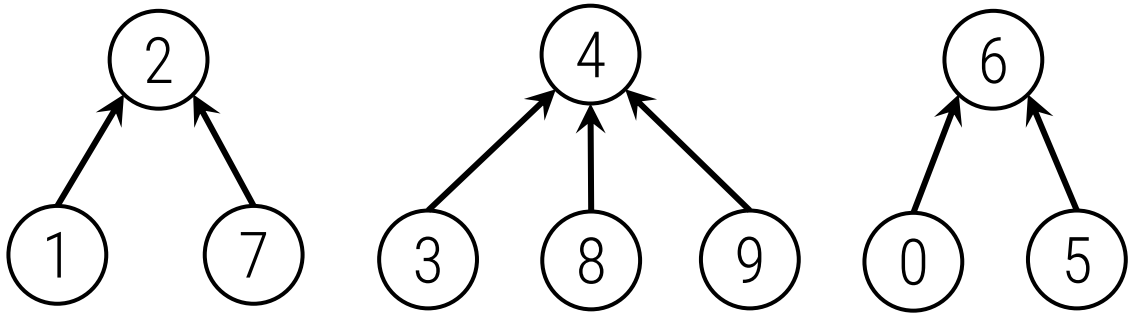
```
unir(7,2)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	2	4	4	6	6	7	4	4

Unión por tamaños

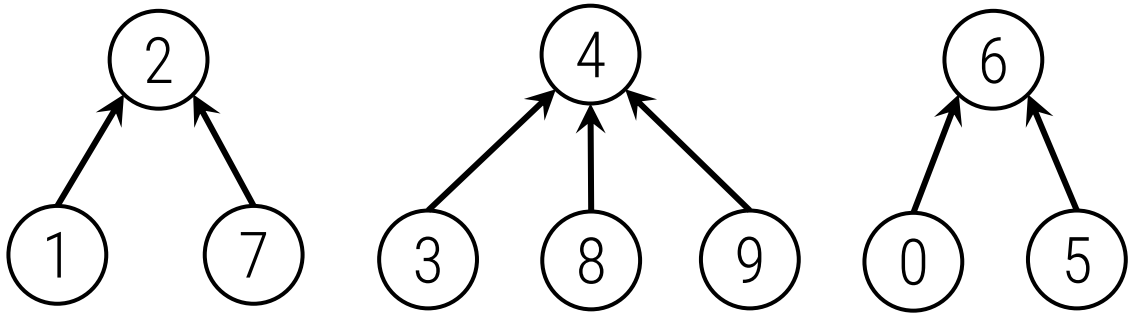
```
unir(7,2)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	2	4	4	6	6	2	4	4

Unión por tamaños

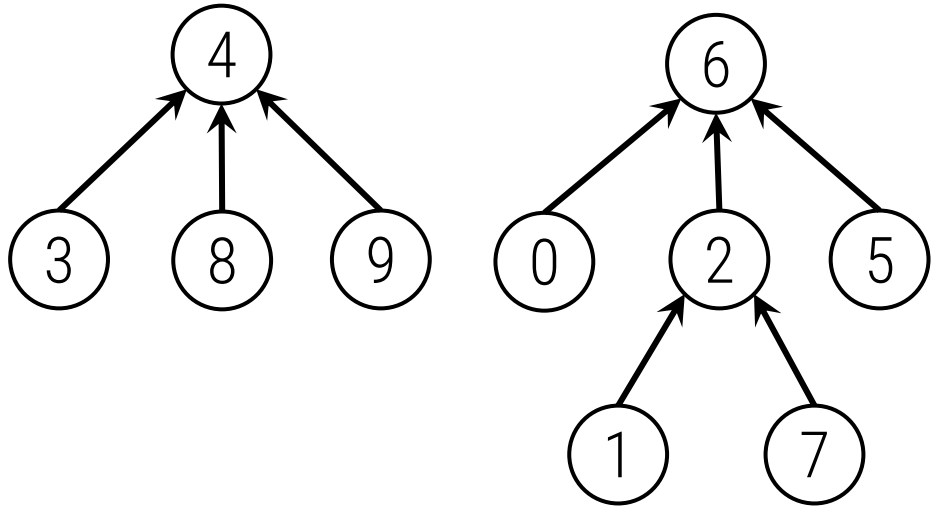
```
unir(6,1)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	2	4	4	6	6	2	4	4

Unión por tamaños

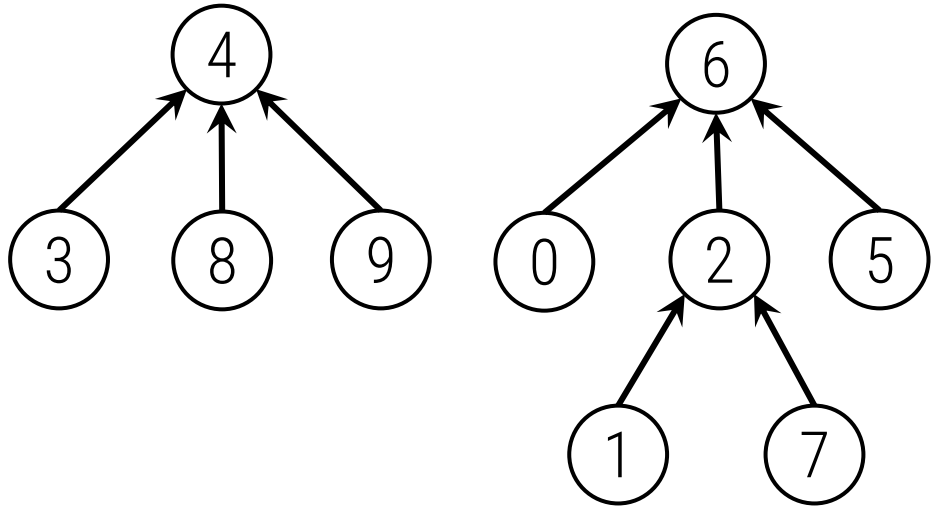
```
unir(6,1)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	6	4	4	6	6	2	4	4

Unión por tamaños

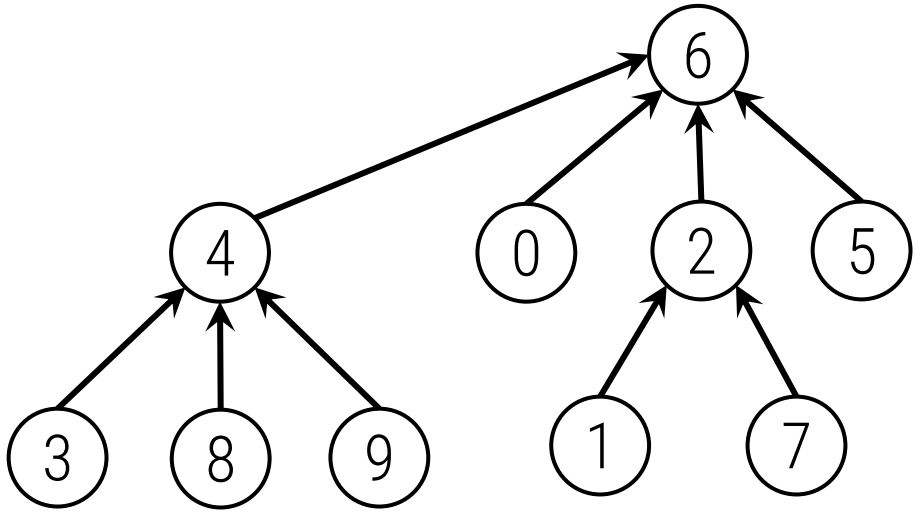
```
unir(7,3)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	6	4	4	6	6	2	4	4

Unión por tamaños

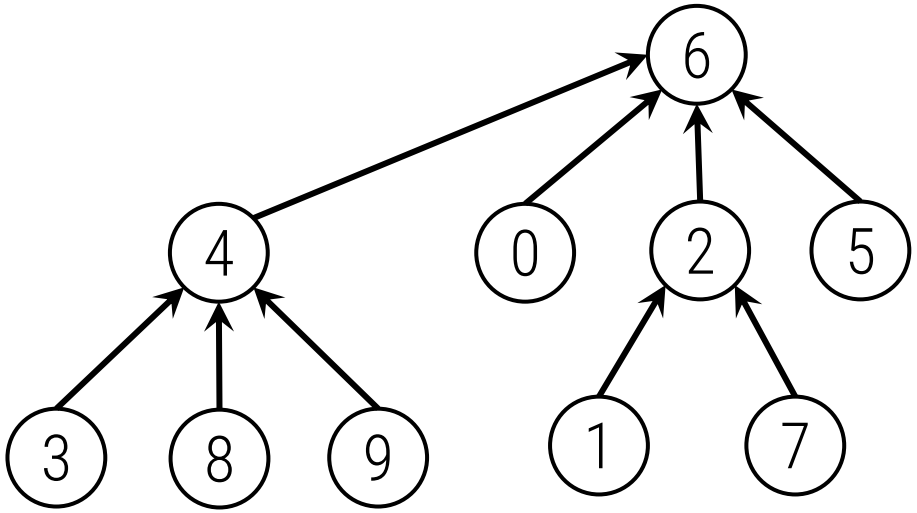
```
unir(7,3)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	6	4	6	6	6	2	4	4

Unión por tamaños

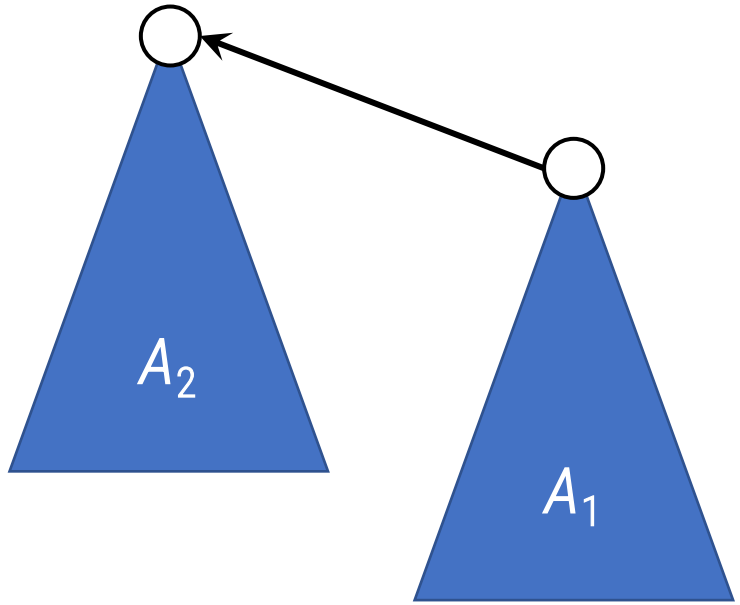
```
unir(7,3)
```



	0	1	2	3	4	5	6	7	8	9
p[]	6	2	6	4	6	6	6	2	4	4

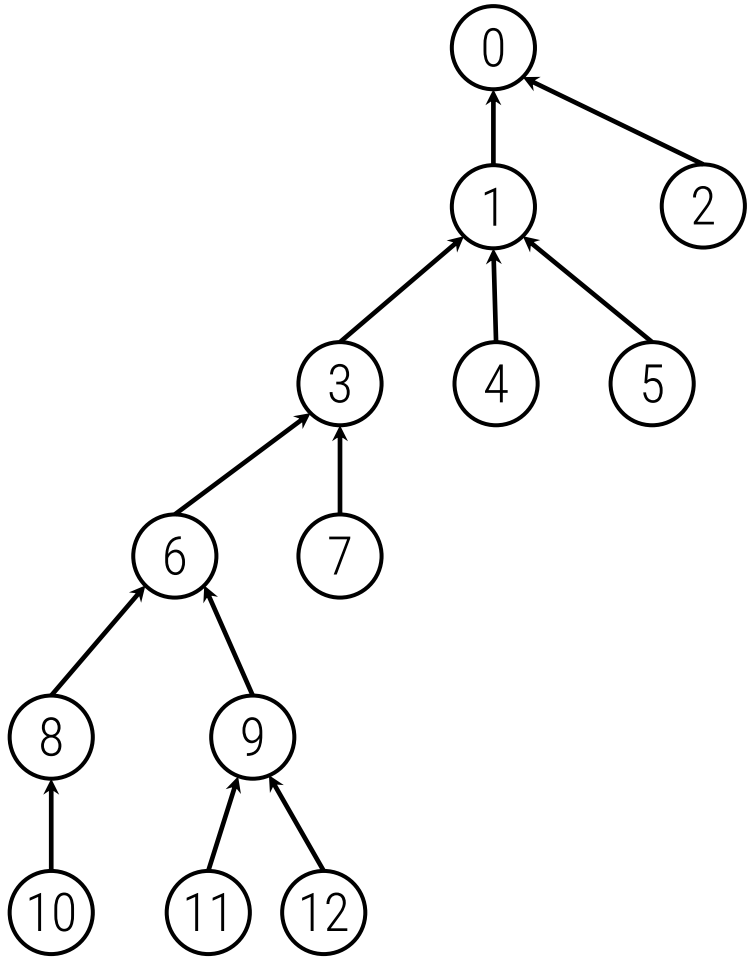
Unión por tamaños, análisis

- La unión de raíces es constante.
- La búsqueda es proporcional a la profundidad del elemento buscado.
- Al hacer la unión por tamaños, la profundidad está acotada por $O(\log N)$.



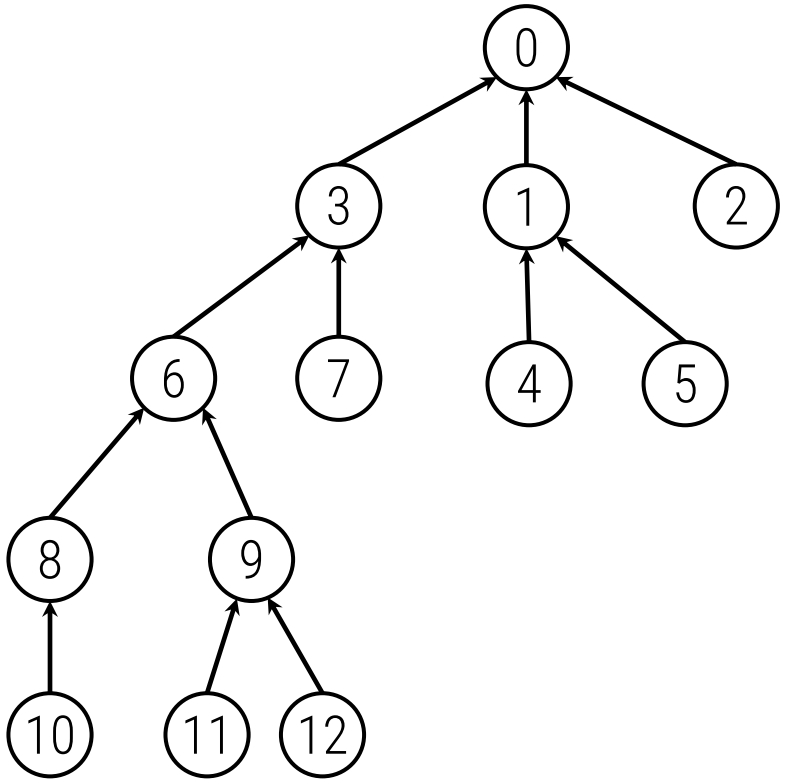
Unión por tamaños y compresión de caminos

- Después de buscar la raíz del árbol donde se encuentra *x*, cambiar su padre para que sea esa raíz.



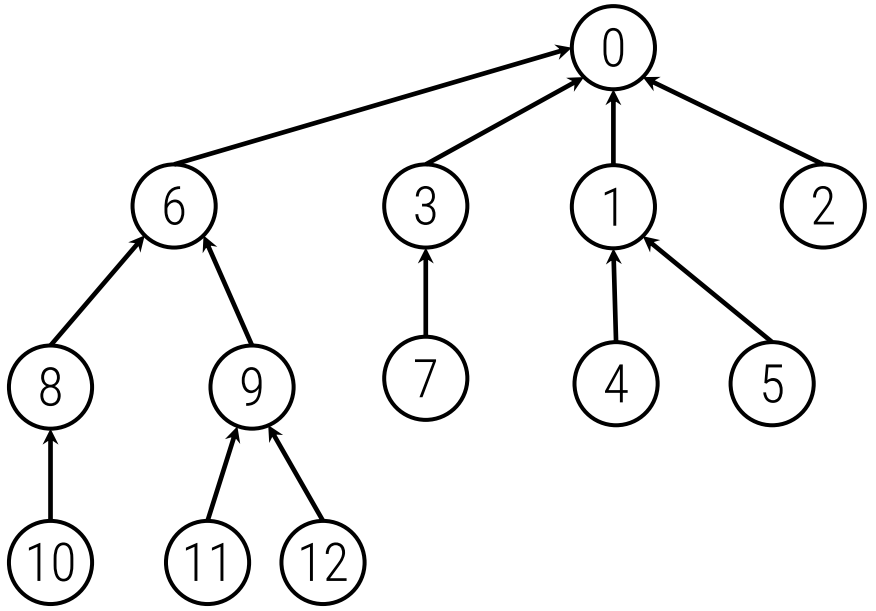
Unión por tamaños y compresión de caminos

- Después de buscar la raíz del árbol donde se encuentra *x*, cambiar su padre para que sea esa raíz.



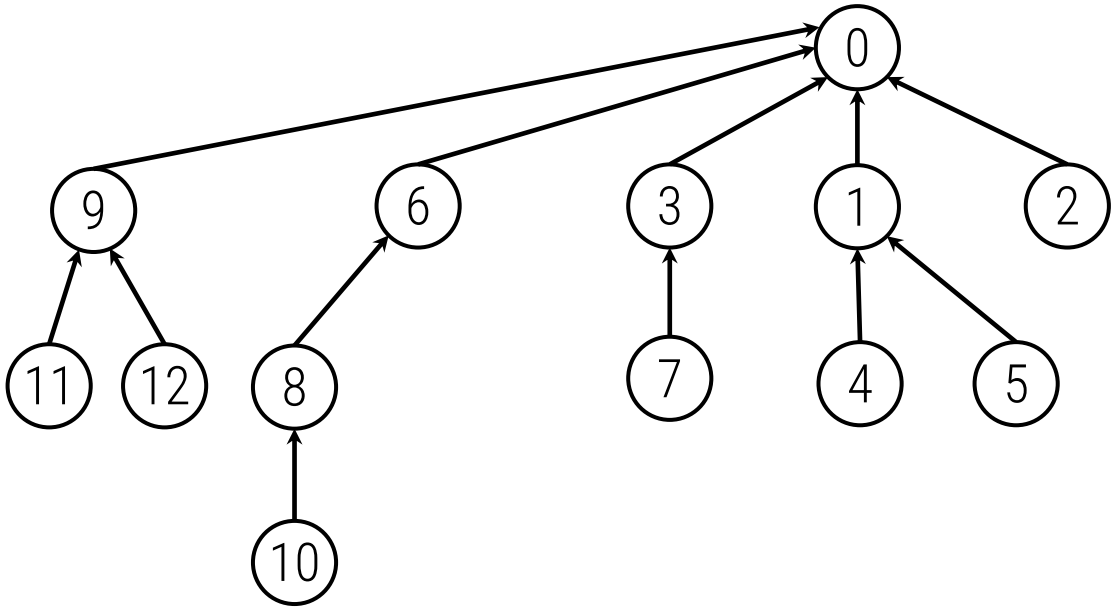
Unión por tamaños y compresión de caminos

- Después de buscar la raíz del árbol donde se encuentra *x*, cambiar su padre para que sea esa raíz.



Unión por tamaños y compresión de caminos

- Después de buscar la raíz del árbol donde se encuentra *x*, cambiar su padre para que sea esa raíz.



Implementación en C++

```
struct ufds {
    int numSets;
    vector<int> p;

    ufds(int N) : numSets(N), p(N, -1) {}

    int find(int i) {
        return (p[i] < 0) ? i : p[i] = find(p[i]);
    }

    bool related(int i, int j) {
        return find(i) == find(j);
    }
```

Implementación en C++

```
    void join(int i, int j) {
        int x = find(i), y = find(j);
        if (x == y) return;
        if (p[x] < p[y])
            swap(x,y);
        p[y] += p[x]; p[x] = y;
        --numSets;
    }

    int size(int i) {
        return -p[find(i)];
    }
};
```